Custom headless ecommerce stack: Nuxt + Medusa + Strapi

A scalable path to a faster customer experience while preserving the existing business flows, SEO structure, subscriptions, upsells, landing pages, and fulfillment logic.

Table Of Contents

1 Executive Summary

2 What We Are Building

3 Technology 1: Nuxt 3 + Vue 3

4 Technology 2: Medusa Commerce

5 Technology 3: Strapi CMS

6 Supporting Infrastructure

7 Why Existing Flows Can Be Replicated

8 Current MVP Status

9 Implemented MVP Flows

10 Suggested Production Structure

11 Hosting Cost Options

12 Traffic And Scaling Assumptions

13 Key Risks And Controls

Use this deck to explain the custom stack option and its infrastructure shape.

Executive Summary

Recommendation: continue building the custom Strapi + Medusa + Nuxt stack. It gives us the control needed to replicate the current website flow while improving speed, stability, and long-term scalability.

MAIN BENEFIT

Full control

We control URLs, checkout, subscriptions, post-purchase upsells, landing pages, fulfillment logic, integrations, and reporting.

BUSINESS GOAL

Faster customer journey

Public pages can be optimized for speed and caching while cart, checkout, account, and subscription pages stay personalized and safe.

TRADEOFF

More ownership

We own hosting, monitoring, releases, backups, and custom integrations. This needs disciplined engineering and support planning.

Summary: this path gives maximum business flexibility, with more engineering responsibility.

Back to TOC

What We Are Building

Nuxt

Customer-facing storefront and application layer.

medusa

Commerce engine for products, carts, checkout, customers, orders, and custom business logic.

strapi

Content platform for landing pages, product content, SEO, articles, and marketing sections.

Customer Browser > **Nuxt Storefront** > **Medusa Commerce**

Content Editors > **Strapi CMS** > **Nuxt Page Rendering**

PostgreSQL | **Redis Cache** | **Cloudflare R2 Media** | **Cloudflare CDN/WAF**

> The storefront combines commerce data from Medusa and content data from Strapi into one fast, SEO-friendly customer experience.

Summary: separate systems own separate responsibilities, then Nuxt composes the final customer-facing page.

Back to TOC

Technology 1: Nuxt 3 + Vue 3

Nuxt is the main website application layer. It is built on **Vue**, a modern frontend framework for interactive customer experiences.

- Renders fast public pages for SEO and performance.
- Builds product pages, landing pages, cart, checkout, account, and post-purchase pages.
- Combines data from Medusa and Strapi before showing it to the customer.
- Controls URL structure, redirects, SEO metadata, and caching strategy.
- Can show server-rendered pages to Google while still supporting rich interactive components for shoppers.
- Lets us keep cart, checkout, account, and post-purchase flows inside one consistent customer experience.

WHY IT MATTERS

Nuxt lets us keep the website fast without giving up control of the customer journey.

It is especially valuable for SEO-heavy ecommerce, landing pages, product pages, and campaign traffic.

Official sites:

nuxt.com

vuejs.org

Summary: Nuxt is the customer-facing engine and the place where speed, SEO, and UX come together.

Back to TOC

Technology 2: Medusa Commerce

medusa

COMMERCE ENGINE

Medusa owns the business data that must be correct for selling products.

- Products, variants, prices, regions, promotions, cart, checkout, customers, orders, and payments.
- Custom modules for subscriptions, post-purchase upsells, fulfillment rules, fraud, and warehouse routing.
- Admin tools for ecommerce managers and customer support workflows.
- Multiple regions and currencies with independent pricing, not just automatic conversion.
- Extensible backend workflows for our current order lifecycle and future integrations.

Medusa gives us a commerce backend that can be extended without forcing our business into a fixed SaaS checkout model.

This is important because our current store has custom subscription, upsell, payment, warehouse, tax, and reporting behavior.

For management, Medusa should be seen as the replacement for the ecommerce engine: it is where product availability, pricing, cart rules, checkout rules, customer data, and order behavior live.

Official site:

medusajs.com

docs.medusajs.com

Summary: Medusa is where custom commerce behavior can be rebuilt under our control.

Back to TOC

Technology 3: Strapi CMS

Strapi is the content management system for the marketing and editorial side of the store.

- Landing pages.
- Product page content sections.
- SEO fields, metadata, FAQs, banners, images, articles, and reusable marketing blocks.
- Content editing by non-developers.
- Role-based access for content and marketing teams.
- Reusable page sections so teams can build pages from approved content blocks.
- Localization support for translated content and market-specific messaging.

MANAGER BENEFIT

Marketing and content teams can update pages without asking developers for every text, image, banner, or landing page change.

Strapi also lets us structure content instead of burying it inside hardcoded templates.

For management, Strapi is the content operations system: it makes the site easier to update while keeping product pricing and checkout logic safely in Medusa.

Official site:

strapi.io

docs.strapi.io

Summary: Strapi gives business teams flexible content control while keeping commerce data separate.

Back to TOC

Supporting Infrastructure

POSTGRESQL

Reliable database

Stores commerce and content data. We use separate databases for Medusa and Strapi so each system has clean ownership.

REDIS

Cache and jobs

Supports caching and background processing. Helps reduce repeated work during high traffic and keeps public pages faster.

CLOUDFLARE R2

Media storage

Stores images and uploaded assets outside the server, so deployments and scaling do not depend on local upload folders.

DOCKER

Repeatable deployment

Packages the storefront, commerce, content, database, and cache services so environments are easier to reproduce.

CLOUDFLARE

CDN and protection

Can cache public pages, reduce origin traffic, and provide security protection before requests reach our servers.

NGINX

Traffic routing

Routes public storefront, Medusa admin/API, and Strapi admin/API to the correct service with HTTPS.

Summary: the stack is modular and scalable, not one large application doing everything.

Back to TOC

Why Existing Flows Can Be Replicated

Because this is a custom stack, we are not limited by a fixed platform checkout or fixed URL model. In practical terms, the current business flows can be replicated 100% if we build and test them intentionally.

- Existing URL structure and SEO paths.
- Product pages and landing pages.
- Cart drawer and cart page behavior.
- Checkout fields and payment rules.
- Post-purchase upsells.
- Subscriptions and account workflows.
- Warehouse routing and tracking updates.
- Loyalty, coupons, store credits, and reporting.
- Fraud, analytics, CRM, and marketing events.
- Country, language, and currency behavior.

Summary: the main risk is implementation effort and QA, not platform limitations.

Back to TOC

Current MVP Status

The first MVP is already deployed to a simple VPS for QA and proof of concept.

- QA storefront is deployed at **particle-turbo.pfm-qa.com**.
- Home page, cart page, and checkout page returned successful live checks.
- Medusa Admin route is reachable through the temporary admin path.
- Strapi is running, with admin path-prefix limitations identified.
- All main containers are running: storefront, commerce, content, two PostgreSQL databases, and Redis.
- HTTPS certificate has been issued.
- Local commerce/content databases were restored to the VPS.

WHAT THIS PROVES

The full stack can run together on a low-cost VPS. This validates the basic architecture before investing in a larger production setup.

The MVP is not the final production architecture, but it proves the first vertical slice works.

Summary: we already have a working QA deployment, not just a theory.

Back to TOC

Implemented MVP Flows

STOREFRONT

Nuxt storefront shell, header, mega menu, mobile menu, footer, original styling tokens, product pages, cart, checkout, and account basics.

CONTENT

Strapi content pages, home sections, product page sections, media through Cloudflare R2, and structured marketing blocks.

COMMERCE

Medusa product data, variant prices, add-to-cart, cart drawer, cart totals, promotions, checkout, Braintree Hosted Fields, and order history basics.

PERFORMANCE

Redis-backed caching for public read-only data like product pages, regular pages, site settings, and recommendations.

PPU

Post-purchase upsell rules, offer page, accept/skip APIs, timeout release job, and Medusa-owned upsell state.

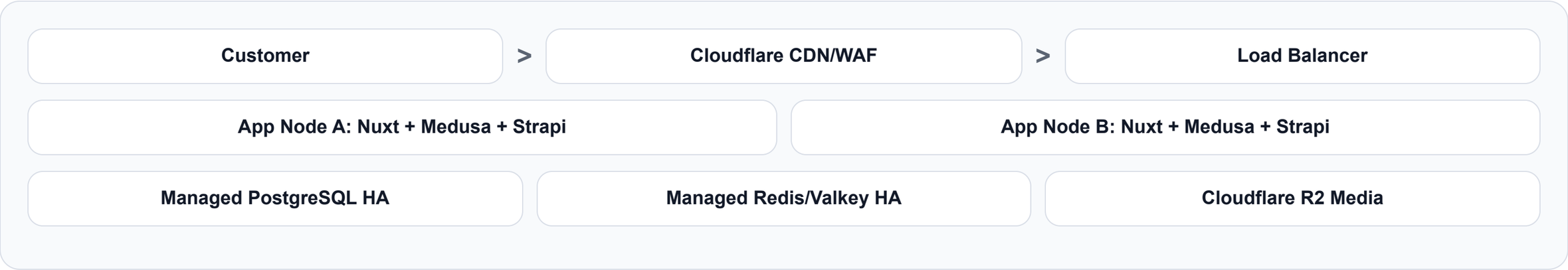
ADMIN TOOLS

Development toolbar and admin links for faster QA while the stack is still in active development.

Summary: core ecommerce flows are already being proven in the new architecture.

Back to TOC

Suggested Production Structure



> Runtime data should not depend on app-node local disk. Databases, cache, and media should be shared services so app nodes can be replaced or scaled safely.

Summary: production should move from one VPS to redundant app nodes plus managed data services.

Back to TOC

Hosting Cost Options

Tier	Monthly Estimate	Shape	Best Use
Minimal Production	~EUR 60-90/month	One VPS with 8-16 vCPU, 32 GB RAM, 500-640 GB NVMe/SSD. Docker runs Nuxt, Medusa, Strapi, PostgreSQL, Redis, and Nginx.	Lowest cost launch or controlled early production. Main risk: one server failure can take down the full store.
Best Middle Ground	~USD 900-1,200/month	Two app nodes, load balancer, managed PostgreSQL HA, managed Redis/Valkey HA, backups, Cloudflare Pro/Business, standard support.	Recommended practical production choice. Removes major single-server risks and supports TV ad bursts.
Best Of The Best	~USD 2,500/month, or ~USD 2,750 with Cloudflare Business	Larger app nodes, larger managed PostgreSQL and Redis, backup/log buffer, monitoring buffer, DigitalOcean Premium Support, optional Cloudflare Business.	Best for major campaigns, high downtime sensitivity, and faster provider response needs.

Summary: the middle-ground option is the best balance of cost, reliability, and scale.

Back to TOC

Traffic And Scaling Assumptions

500/min
Regular expected traffic, about 8 requests per second.

1,500-3,000/min
TV ad spike range, about 25-50 requests per second.

2 nodes
Recommended production app layer for high availability.

HA data
Managed database and cache reduce the biggest outage risks.

> Public product and landing pages should be aggressively cache-friendly. Cart, checkout, account, and subscription pages should remain dynamic and customer-specific.

Summary: scaling comes from caching public pages, separating data services, and adding app nodes behind a load balancer.

Back to TOC

Key Risks And Controls

RISKS

- More engineering ownership than a SaaS platform.
- Custom flows need complete QA before launch.
- Production monitoring and backups must be treated as business-critical.
- Data migration for customers, orders, subscriptions, URLs, and content must be rehearsed.
- Payment and fulfillment integrations need end-to-end testing.

CONTROLS

- Use managed database and cache for production.
- Keep media in R2, not on app servers.
- Use Cloudflare for CDN, WAF, and traffic offload.
- Maintain staging environment before every release.
- Run launch rehearsals and rollback planning.

Summary: the custom path is controllable if operations are planned from the start.

Back to TOC